

CSA3023/CSE3023/CSM3023 WEB-BASED APPLICATION DEVELOPMENT

Lab Modules



WAN NURAL JAWAHIR HJ WAN YUSSOF, RABIEI MAMAT,
MOHD ARIZAL SHAMSIL MAT RIFIN
& MUHAMMAD SYAFFIQ HAZARI

Contents

List of Figures

iv

1	DEVELOPMENT ENVIRONMENT SETUP FOR JAVA WEB APPLICATIONS	1
1.1	Objectives	1
1.2	Introduction	1
1.3	Software Requirements	2
1.4	Task 1: Install Java Development Kit (JDK)	2
1.4.1	Step 1: Download JDK	2
1.4.2	Step 2: Install JDK	2
1.4.3	Step 3: Verify JDK Installation	2
1.5	Task 2: Install XAMPP	3
1.5.1	Step 1: Download XAMPP	3
1.5.2	Step 2: Install XAMPP without Tomcat	4
1.5.3	Step 3: Verify XAMPP Installation	9
1.6	Task 3: Change the Default Root Password of MySQL	11
1.6.1	Step 1: Start Apache and MySQL Services	11
1.6.2	Step 2: Run Command	12
1.7	Task 4: Tomcat 9 Installation	18
1.7.1	Step 1: Download Apache Tomcat 9	18
1.7.2	Step 2: Extract Apache Tomcat 9	19
1.7.3	Step 3: Start Tomcat Server	19
1.7.4	Step 4: Verify Installation	20
1.8	Task 5: NetBeans IDE Installation	21
1.8.1	Step 1: Download Apache NetBeans 29	21
1.8.2	Step 2: Extract and Install NetBeans 29	21
1.8.3	Step 3: Configure NetBeans 29	23
1.8.4	Step 5: Configure Tomcat in NetBeans	23
1.9	Task 6: Writing a Simple Java Servlet	26
1.9.1	Step 1: Create New Project	26
1.9.2	Step 2: Create New Servlet	29
1.9.3	Step 3: Run Servlet	32
1.10	Task 7: Writing a Simple JSP Program	34
1.10.1	Step 1: Create a New Project	34
1.10.2	Step 2: Create a New JSP File	36
1.10.3	Step 3: Run JSP File	39
1.11	Task 8: Use Java Reference Datatype/Class Wrapper in JSP	41
1.11.1	Step 1: Create a New JSP File	41

1.12	Exercise	43
2	SERVLET: DATA SHARING AND DATABASE MANAGEMENT	45
2.1	Objectives	45
2.2	Introduction	46
2.3	Software Requirements	46
2.4	Task 1: Data Sharing in Servlet	46
2.4.1	Step 1: Project Setup	46
2.4.2	Step 2: Create HTML Login Page	48
2.4.3	Step 3: Create a Login Servlet	48
2.4.4	Step 4: Create an Account Servlet	51
2.4.5	Step 5: Run the Project	52
2.5	Task 2: Creating A Table in MySQL Database	53
2.5.1	Step 1: Start Apache and MySQL	53
2.5.2	Step 2: Create a New Database	53
2.5.3	Step 3: Create a New MySQL Table	53
2.6	Task 3: Setting the Environment of Web Application for Database Connection	54
2.6.1	Step 1: Create a New Web Application Project	54
2.6.2	Step 2: Prepare the MySQL Connector	55
2.7	Task 4: Using Servlets for Database CRUD Operations	57
2.7.1	Step 1: Create HTML Form for Data Entry	57
2.7.2	Step 2: Create the Insert Servlet (Create)	58
2.7.3	Step 3: Create the View Servlet (Read)	59
2.7.4	Step 4: Create the Delete Servlet (Delete)	62
2.7.5	Step 5: Create the Update Servlet (Update)	63
2.8	Task 5: Using Servlets, DAO and Java Beans for Database CRUD Operations	65
2.8.1	Step 1: Copy and Rename the Project	66
2.8.2	Step 2: Create the User JavaBean (Model)	66
2.8.3	Step 3: Create the Data Access Object (UserDAO)	68
2.8.4	Step 4: Modify the Existing Servlets (Controller)	72
2.9	Exercise	77
3	JSP: SCRIPTLET, PAGE DIRECTIVE & INCLUDE DIRECTIVE	79
3.1	Objectives	79
3.2	Introduction	79
3.3	Software Requirements	80
3.4	Task 1: Passing Data from Main JSP's Page to Other JSP's Page	80
3.4.1	Step 1: Create A New JSP Page	80
3.4.2	Step 2: Write an HTML Markup	81
3.5	Task 2: Using Mathematics Operations in JSP	84
3.5.1	Step 1: Create A New JSP Page	85
3.5.2	Step 2: Write an HTML Markup	85
3.5.3	Step 3: Run the <code>Calculator.jsp</code>	86
3.6	Task 3: Populate an Array Values into HTML's Table	89
3.6.1	Step 1: Create A New JSP Page	89
3.6.2	Step 2: Write An HTML Markup	89
3.6.3	Step 3: Run the <code>populateArray.jsp</code> File	90

3.7	Task 4: Perform Calculation of Car Loan	90
3.7.1	Step 1: Create A New HTML File	90
3.7.2	Step 2: Create An HTML Markup	91
3.7.3	Step 3: Create <code>processCalculateCarLoan.jsp</code>	92
3.8	Task 5: Using JSP Page Directive to Call Java API	93
3.9	Task 6: Use JSP Include directive for JSP Page	98
4	JSP: SCRIPTLET, EXPRESSION & STANDARD ACTIONS	103
4.1	Objectives	103
4.2	Introduction	103
4.3	Software Requirements	104
4.4	Task 1: Using JSP Scripting	110
4.4.1	Step 1: Create <code>customer.html</code>	110
4.4.2	Step 2: Create <code>processCustomer.jsp</code>	111
4.4.3	Step 3: Compile and Run the Application	113
4.5	Task 2: Using JSP (Scripting, Declaration and Expression)	115
4.5.1	Step 1: Create <code>currencyConversion.html</code>	115
4.5.2	Step 2: Create <code>processCurrency.jsp</code>	116
4.5.3	Step 3: Compile and Run the Application	118
4.6	Task 3: Using JSP Standard Action (Include and Param)	120
4.6.1	Step 1: Create <code>jspParameter.jsp</code>	120
4.6.2	Step 2: Create <code>subjectInfo.jsp</code>	122
4.6.3	Step 3: Compile and Run the Application	123
4.7	Task 4: Using JSP Standard Action (Forward)	124
4.7.1	Step 1: Create <code>forward.jsp</code>	124
4.7.2	Step 2: Create <code>forwardInfo.jsp</code>	125
4.7.3	Step 3: Compile and Run the Application	126
4.8	Task 5: Using JSP Scriptlet to Construct Business Logic	127
4.8.1	Step 1: Create Project and JSP File <code>insuranceQuotation.jsp</code>	127
4.8.2	Step 2: Design Insurance Quotation Interface	127
4.8.3	Step 3: Create <code>processInsuranceQuo.jsp</code>	128
4.8.4	Step 4: Implement Business Logic Using JSP Scriptlet	129
4.8.5	Step 5: Compile and Run the Application	130

List of Figures

1.1	XAMPP Download Platform	3
1.2	XAMPP Setup Wizard Startup Screen	5
1.3	XAMPP Select Components	5
1.4	XAMPP Installation Directory	6
1.5	XAMPP Language	7
1.6	XAMPP Installation Ready	7
1.7	XAMPP Installation Progress	8
1.8	XAMPP Installation Finish	8
1.9	XAMPP Control Panel	9
1.10	XAMPP Control Panel Running	10
1.11	XAMPP Dashboard	10
1.12	phpMyAdmin Interface	11
1.13	XAMPP Control Panel-Click Shell	11
1.14	Verify Password	12
1.15	Error Page	13
1.16	Apache Tomcat9 Download Page	18
1.17	Start Tomcat Server	19
1.18	Apache Tomcat Homepage	20
1.19	Apache NetBeans 29 Download Page	21
1.20	Extract netbeans-29-bin.zip	22
1.21	Netbean Executable File	22
1.22	Netbean IDE GUI	23
1.23	Configure Server	24
1.24	Add Server	24
1.25	Add Server	25
1.26	Add Server	25
1.27	Add Server	26
1.28	Create New Project	27
1.29	Create New Web Application	27
1.30	Enter Project Details	28
1.31	Create New Servlet	29
1.32	Name the Servlet	30
1.33	Configure Servlet Deployment	30
1.34	HelloServlet.java File	31
1.35	<code>processRequest()</code> Method	31
1.36	Modified <code>processRequest()</code> Method	31
1.37	Run the Servlet	32

1.38	Dialog Box Call to /HelloServlet	32
1.39	Output	33
1.40	Modified <code>doGet()</code> Method	33
1.41	Dialog Box with Passing Parameter	34
1.42	Output	34
1.43	Create New Web Application	35
1.44	Enter Project Details	36
1.45	Configure the Server	36
1.46	Create a New JSP File	37
1.47	Enter Name and Location	37
1.48	Modified JSP File	38
1.49	Compile JSP File	38
1.50	Compile JSP File	38
1.51	Run JSP File	39
1.52	Output	40
1.53	Start/Stop Apache Tomcat	40
1.54	Create <code>useJavaObject.jsp</code>	41
1.55	<code>useJavaObject.jsp</code>	42
1.56	Output	43
2.1	Create New Servlet	49
2.2	Create Login Servlet	49
2.3	Configure servlet deployment	50
2.4	Creating the “CRUDusingServlet” Project	55
2.5	Downloading MySQL Connector/J	55
2.6	Extracting the MySQL Connector/J Archive	56
2.7	Adding External Library to Project	56
2.8	Importing MySQL Connector/J Library	57
3.1	Create new JSP	81
3.2	New JSP	81
3.3	<code>memberRegister.jsp</code> Interface	82
3.4	Filling input <code>memberRegister.jsp</code>	84
3.5	Output in <code>memberProcessing.jsp</code>	84
3.6	Create new JSP	85
3.7	New JSP	85
3.8	<code>calculator.jsp</code> Interface	86
3.9	Key-in Filename	89
3.10	Output from Array	90
3.11	Creating New HTML File	91
3.12	Interface Output for Car Loan Calculation HTML	91
3.13	Input for Car Loan Calculation	92
3.14	Output for Car Loan Calculation	93
3.15	New JSP	93
3.16	Key-in Filename	94
3.17	Save All	96
3.18	Run	96
3.19	Success Message	96

3.20	ArrayList Output	97
3.21	Tomcat Before Run	97
3.22	Tomcat After Run	97
3.23	Tomcat Success Message	98
3.24	mainPage.jsp	98
3.25	headerPage.jsp Output	98
3.26	footerPage.jsp Output	99
3.27	mainPage.jsp Output	99
4.1	Creating customer.html	111
4.2	Interface of customer.html	111
4.3	Creating processCustomer.jsp	112
4.4	Example input in customer.html	114
4.5	Output after processing customer input	114
4.6	Creating currencyConversion.html	115
4.7	Interface of currencyConversion.html	116
4.8	Creating processCurrency.jsp	116
4.9	Interface of currencyConversion.html	119
4.10	Input example in currencyConversion.html	119
4.11	Processing result of currency conversion	120
4.12	Creating jspParameter.jsp	121
4.13	Creating subjectInfo.jsp	122
4.14	Output of jspParameter.jsp	123
4.15	Creating forward.jsp	124
4.16	Creating forwardInfo.jsp	125
4.17	Output of forwarded information	126
4.18	Creating insuranceQuotation.jsp	127
4.19	Insurance Quotation Interface	128
4.20	Creating processInsuranceQuo.jsp	128
4.21	Insurance Quotation Output	131

Lab Module 4

JSP: SCRIPTLET, EXPRESSION & STANDARD ACTIONS

4.1 Objectives

By the end of this lab, students should be able to:

1. Apply JSP Scriptlet and Expression elements to process input data and generate dynamic web content.
2. Utilize JSP Declaration to define variables, constants, and methods for reusable logic within JSP pages.
3. Implement JSP Standard Actions such as `<jsp:include>` and `<jsp:param>` to enable modular design and data sharing between pages.
4. Control request flow and page navigation using `<jsp:forward>`.
5. Develop and integrate business logic within JSP applications using Java Scriptlets.
6. Design and develop a complete JSP-based web application by integrating scripting elements, standard actions, and dynamic data processing.

4.2 Introduction

This lab introduces the fundamental concepts of **JavaServer Pages (JSP)** for developing dynamic web applications. Students will explore how JSP enables the integration of Java code within HTML to process user input, perform computations, and generate dynamic content. Key JSP elements such as **Scriptlets**, **Expressions**, and **Declarations** are used to implement core programming logic directly within web pages, allowing students to understand how server-side processing works in a web environment.

In addition, the lab emphasizes the use of **JSP Standard Actions** including `jsp:include`, `jsp:param`, and `jsp:forward` to support modular design, data sharing, and page navigation. Through a series of structured tasks, students will progressively build their skills from simple data processing to implementing complete web-based applications with integrated business logic. By the end of the lab, students will be able to develop structured, reusable, and interactive JSP applications that follow good web development practices.

4.3 Software Requirements

Software	Website
Java JDK	https://www.oracle.com/java/technologies/downloads/
XAMPP	https://www.apachefriends.org
Apache Tomcat	https://tomcat.apache.org

Note: Students are allowed to use the provided `style.css` below to achieve a similar interface design as shown in the figures. However, the use of this CSS file is **not compulsory**. Students may also design their own interface layout as long as the required functionality is implemented correctly.

```
/* Basic Settings */
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f4f7f6;
    margin: 0;
    padding: 20px;

    /* REMOVE FLEX */
    display: block;
}
```

```

.container {
    width: 100%;
    max-width: 900px;
    margin: 0 auto; /* center properly */
}

/* Main Title */
h1 {
    font-size: 1.5rem;
    color: #333;
    margin-bottom: 20px;
    border-left: 5px solid #6f42c1;
    padding-left: 10px;
}

/* Form Card */
.card {
    background: #ffffff;
    padding: 30px;
    border-radius: 8px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.1);
}

/* Form Title (Purple Heading) */
.form-title {
    color: #6f42c1;
    font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
    font-size: 1.8rem;
    font-weight: 600;
    letter-spacing: -0.5px;
    margin-top: 0;
    margin-bottom: 25px;
    text-transform: capitalize;
}

/* Input & Form Group Styling */
.form-group {
    margin-bottom: 20px;
}

label {

```

```

    display: block;
    font-weight: bold;
    margin-bottom: 8px;
    color: #555;
}

input[type="text"],
input[type="number"] {
    width: 100%;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
    box-sizing: border-box;
}

/* Radio Button Styling */
.radio-group {
    display: flex;
    gap: 20px;
    align-items: center;
}

.radio-label {
    font-weight: normal;
    cursor: pointer;
    display: flex;
    align-items: center;
    gap: 5px;
}

/* Button Styling */
.button-group {
    margin-top: 30px;
    display: flex;
    gap: 10px;
}

.btn {
    padding: 10px 25px;
    border: none;
    border-radius: 4px;

```

```
    cursor: pointer;
    font-weight: bold;
    transition: 0.3s;
}

.btn-submit {
    background-color: #e0e0e0;
    color: #333;
    border: 1px solid #adadad;
}

.btn-submit:hover {
    background-color: #d0d0d0;
}

.btn-cancel {
    background-color: #e0e0e0;
    color: #333;
    border: 1px solid #adadad;
}

.btn-cancel:hover {
    background-color: #ffcccc;
}

/* Dropdown Select Styling */
select {
    width: 100%;
    padding: 12px;
    border: 1px solid #ced4da;
    border-radius: 6px;
    background-color: white;
    font-size: 1rem;
    cursor: pointer;
}

select:focus {
    outline: none;
    border-color: #6f42c1;
}
```

```

/* ===== RESULT PAGE STYLES (TWO-COLUMN LAYOUT) ===== */
.result-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
    margin-bottom: 25px;
}

.result-box {
    background: #f9f9fb;
    padding: 20px;
    border-radius: 8px;
    border: 1px solid #e0e0e0;
}

.result-box h3 {
    margin-top: 0;
    margin-bottom: 20px;
    color: #6f42c1;
    font-size: 1.3rem;
    border-bottom: 2px solid #6f42c1;
    padding-bottom: 10px;
}

/* Result text styling */
.result-group {
    margin-bottom: 15px;
    padding: 8px 0;
    border-bottom: 1px dashed #e0e0e0;
}

.result-group:last-child {
    border-bottom: none;
}

.result-group label {
    display: inline-block;
    font-weight: 600;
    color: #555;
    min-width: 140px;
}

```

```

.result-group p {
    display: inline-block;
    margin: 0;
    color: #333;
    font-weight: 500;
}

/* Alternative Flexbox layout for result groups */
.result-group-flex {
    display: flex;
    justify-content: space-between;
    margin-bottom: 12px;
    padding: 8px 0;
    border-bottom: 1px dashed #e0e0e0;
}

.result-group-flex label {
    font-weight: 600;
    color: #555;
}

.result-group-flex p {
    margin: 0;
    color: #333;
    font-weight: 500;
}

/* Back Button Styling (Result Page) */
.btn-back {
    background-color: #6f42c1;
    color: white;
    border: none;
    padding: 10px 25px;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
    transition: 0.3s;
}

.btn-back:hover {

```

```

    background-color: #5a32a3;
}

/* Responsive Design for Mobile */
@media (max-width: 768px) {
    .result-grid {
        grid-template-columns: 1fr;
        gap: 15px;
    }

    .result-group label {
        display: block;
        margin-bottom: 5px;
    }

    .result-group p {
        display: block;
        margin-left: 0;
    }

    .result-group-flex {
        flex-direction: column;
    }
}

```

4.4 Task 1: Using JSP Scripting

Objective: Apply JSP Scriptlet and JSP Expression to process input data and display dynamic output.

4.4.1 Step 1: Create customer.html

1. Create a new project named **Lab4**.
2. Create a new HTML file and name it `customer.html`.
3. Refer to Figure 4.1 for file creation.

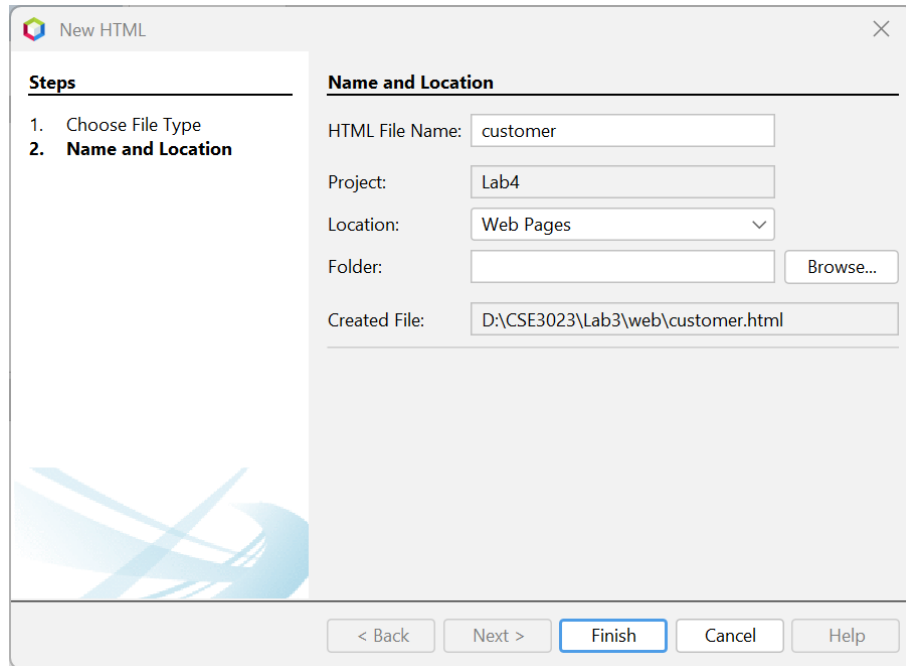


Figure 4.1: Creating `customer.html`

4. Design the graphical user interface (GUI) as shown in Figure 4.2.

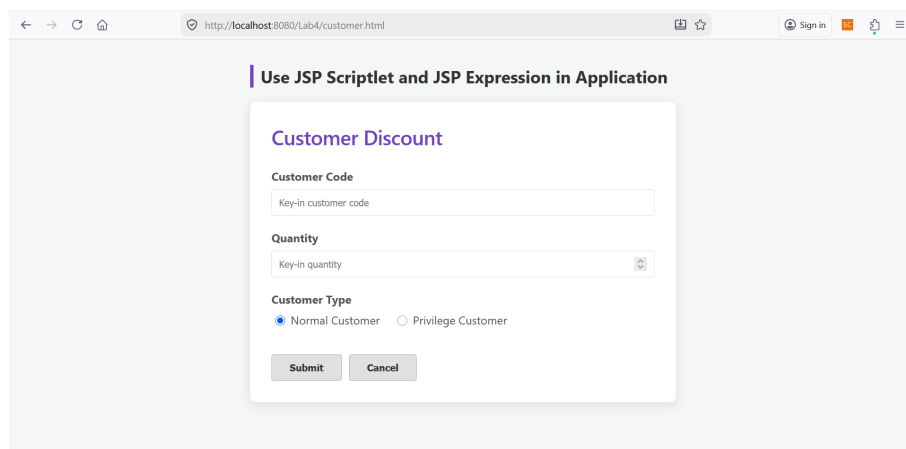


Figure 4.2: Interface of `customer.html`

5. Ensure that the **Quantity** field accepts numeric input only.

6. Assign the following values for customer type:

- Normal Customer = “1”
- Privilege Customer = “2”

4.4.2 Step 2: Create `processCustomer.jsp`

1. Create a new JSP file named `processCustomer.jsp`.

2. Refer to Figure 4.3 for file creation.

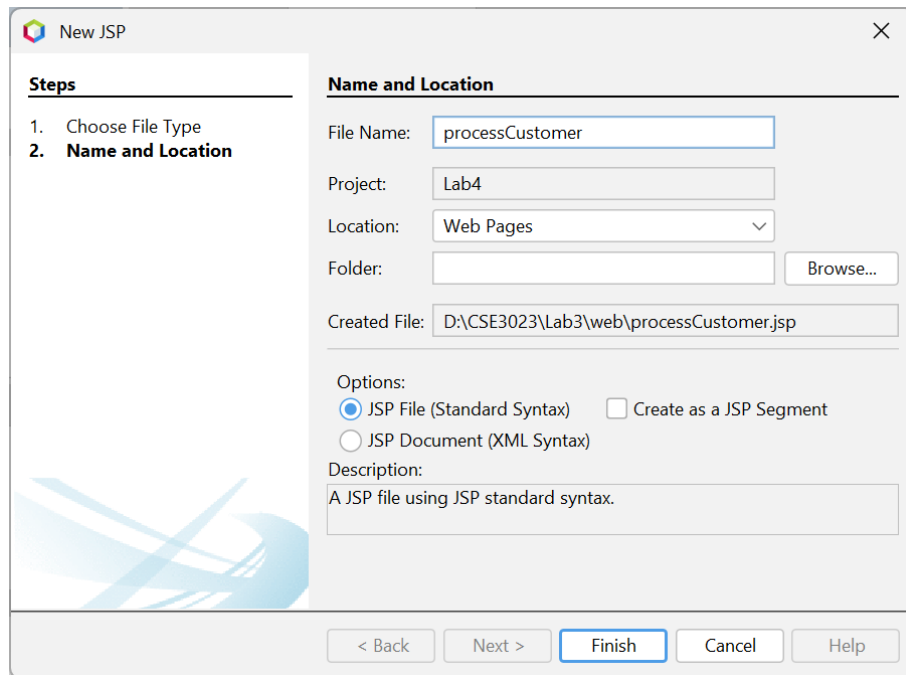


Figure 4.3: Creating processCustomer.jsp

3. Implement JSP Scriptlet to:

- Retrieve input values using `request.getParameter()`
- Convert input values to appropriate data types
- Apply business logic to calculate discount and total amount

4. Use the following code as a reference:

```
<%  
    // Fixed price  
    final double price = 10.0;  
  
    // Retrieve form data  
    String cust_no = request.getParameter("customerCode");  
    String cust_type = request.getParameter("customerType");  
  
    int quantity = 0;  
    try {  
        quantity = Integer.parseInt(request.getParameter("quantity"));  
    } catch (Exception e) {  
        quantity = 0;  
    }  
}
```

```

// Business logic
double total = 0;
String message = "";

if (cust_type.equals("1") && quantity > 100) {
    message = "You're entitled to 10% discount";
    total = quantity * price * 0.9;

} else if (cust_type.equals("2") && quantity > 100) {
    message = "You're entitled to 25% discount";
    total = quantity * price * 0.75;

} else {
    message = "You're not entitled to any discount";
    total = quantity * price;
}

// Display customer type
String custTypeDisplay = cust_type.equals("1") ?
    "Normal Customer" : "Privilege Customer";
%>

```

5. Use **JSP Expression** to display the processed output (e.g., customer details, discount status, and total amount).

4.4.3 Step 3: Compile and Run the Application

1. Compile both `customer.html` and `processCustomer.jsp`.
2. Run `customer.html` in a web browser.
3. Enter the required information as shown in Figure 4.4.

← → ↻ ⌂ http://localhost:8080/Lab4/customer.html Sign in

Use JSP Scriptlet and JSP Expression in Application

Customer Discount

Customer Code
S1234

Quantity
101

Customer Type
☐ Normal Customer ☒ Privilege Customer

Figure 4.4: Example input in `customer.html`

4. Observe the output displayed in the browser as shown in Figure 4.5.

← → ↻ ⌂ http://localhost:8080/Lab4/processCustomer.jsp 90% Sign in

Customer Discount Result

Transaction Summary

Customer Code:
S1234

Quantity:
101

Customer Type:
Privilege Customer

Status:
You're entitled to 25% discount

Total Amount:
RM 757.50

Figure 4.5: Output after processing customer input

Reflection

1. What have you learnt from this exercise?
2. Explain three (3) types of JSP scripting elements.

4.5 Task 2: Using JSP (Scripting, Declaration and Expression)

Objective: Apply JSP Declaration, JSP Scriptlet, and JSP Expression to perform currency conversion in a web application.

4.5.1 Step 1: Create currencyConversion.html

1. Create a new project named **Lab4**.
2. Create a new HTML file and name it `currencyConversion.html`.
3. Refer to Figure 4.6 for file creation.

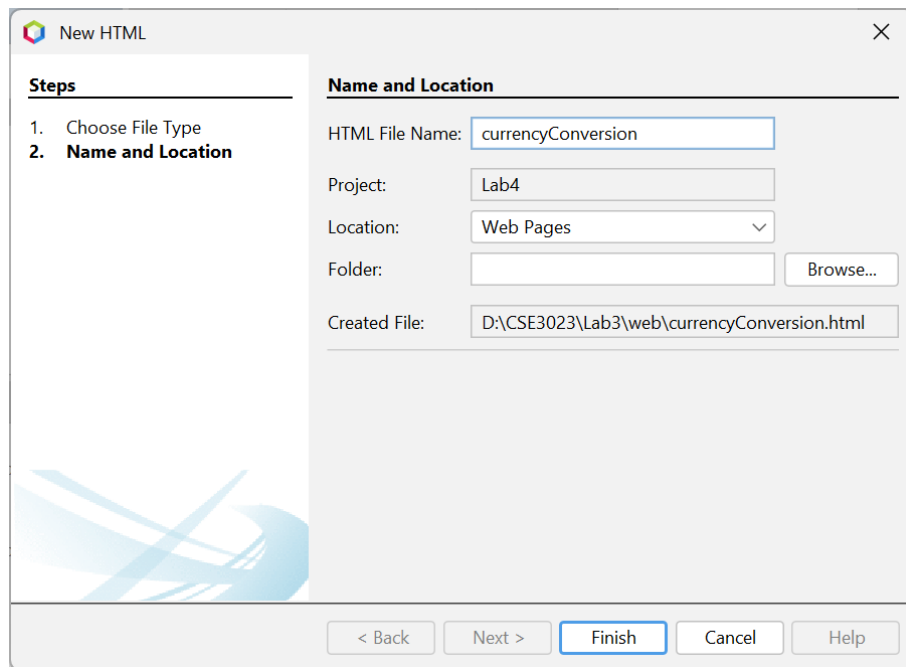


Figure 4.6: Creating `currencyConversion.html`

4. Design the graphical user interface (GUI) as shown in Figure 4.7.
5. Ensure that the **Amount** field accepts numeric input only.
6. Assign the following values for currency selection:
 - USD = “1”
 - EURO = “2”
 - JPY = “3”
 - SGD = “4”

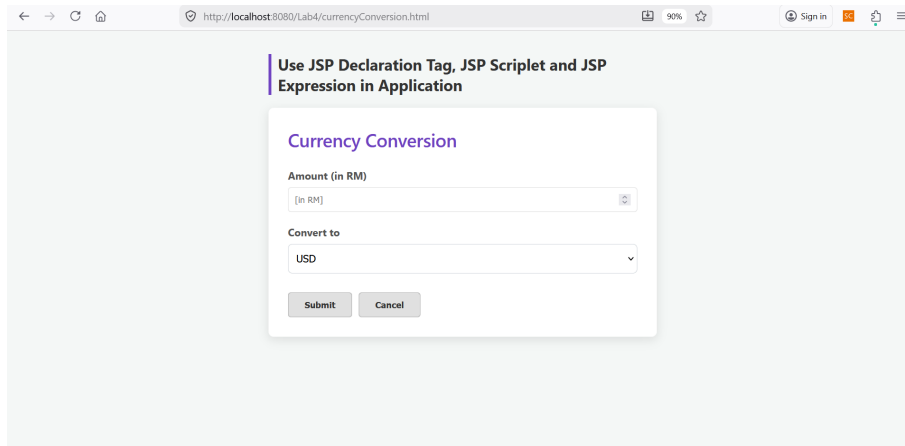


Figure 4.7: Interface of `currencyConversion.html`

4.5.2 Step 2: Create `processCurrency.jsp`

1. Create a new JSP file named `processCurrency.jsp`.
2. Refer to Figure 4.8 for file creation.

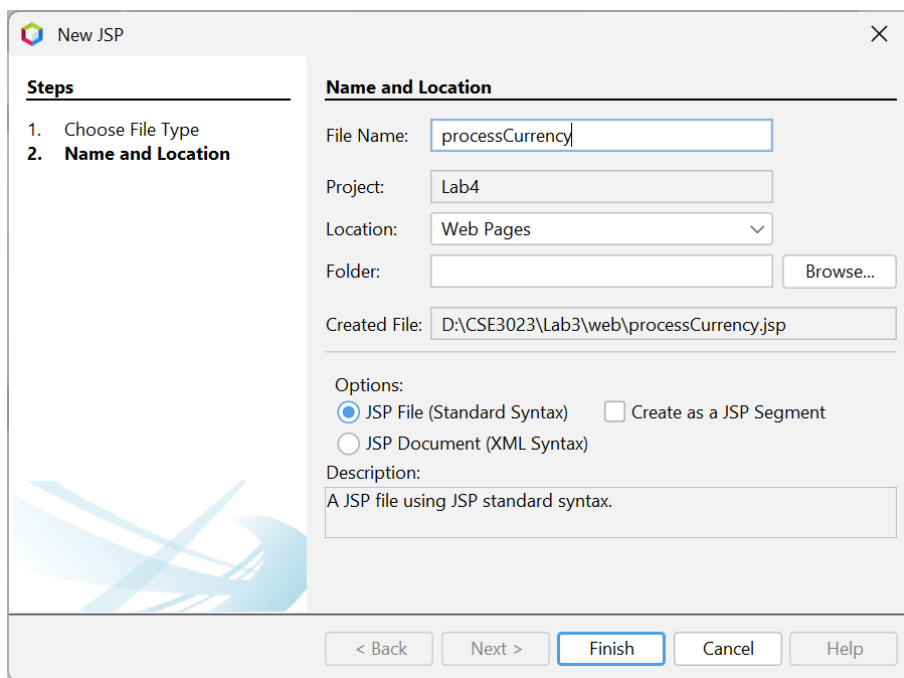


Figure 4.8: Creating `processCurrency.jsp`

3. Use the **JSP Declaration tag** (`<%! ... %>`) to:
 - Define currency rates as constants
 - Create a reusable method `calculateRate()`
4. Use the following code as a reference:

```

<%!
    // Define constant exchange rates
    final double USD = 0.25;
    final double EURO = 0.21;
    final double JPY = 40;
    final double SGD = 0.32;

    // Method to calculate conversion
    private double calculateRate(String currency, int amount) {
        double currencyChange = 0.0;

        if (currency != null) {
            if (currency.equals("1")) {
                currencyChange = amount * USD;
            } else if (currency.equals("2")) {
                currencyChange = amount * EURO;
            } else if (currency.equals("3")) {
                currencyChange = amount * JPY;
            } else {
                currencyChange = amount * SGD;
            }
        }
        return currencyChange;
    }
%>

```

5. Use **JSP Scriptlet** (<% ... %>) to:

- Retrieve input values using `request.getParameter()`
- Convert input into numeric data type
- Call the `calculateRate()` method

```

<%
    String currencyType = request.getParameter("currencyType");
    String amountRaw = request.getParameter("amount");

    int amount = 0;
    double total = 0;

    try {

```

```

        if (amountRaw != null) {
            amount = Integer.parseInt(amountRaw);
            total = calculateRate(currencyType, amount);
        }
    } catch (Exception e) {
        amount = 0;
    }

    // Determine currency name for display
    String currencyName = "N/A";
    if ("1".equals(currencyType)) currencyName = "USD";
    else if ("2".equals(currencyType)) currencyName = "EURO";
    else if ("3".equals(currencyType)) currencyName = "JPY";
    else if ("4".equals(currencyType)) currencyName = "SGD";
%>

```

6. Use **JSP Expression** (<%= ... %>) to display the output:

```

<label>Amount in Ringgit Malaysia (RM):</label>
<p>RM <%= amount %></p>

<label>Converted to (<%= currencyName %>):</label>
<p><%= String.format("%.2f", total) %></p>

```

4.5.3 Step 3: Compile and Run the Application

1. Compile both currencyConversion.html and processCurrency.jsp.
2. Run currencyConversion.html.

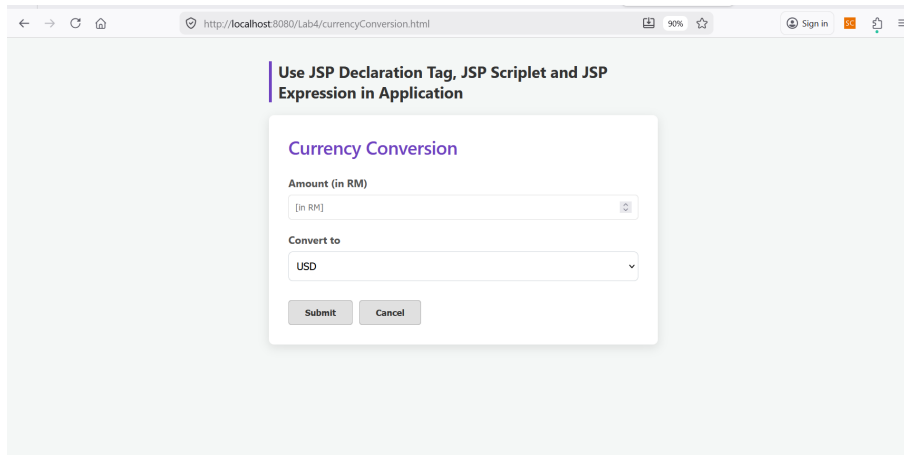


Figure 4.9: Interface of `currencyConversion.html`

3. Enter input data as shown in Figure 4.9.

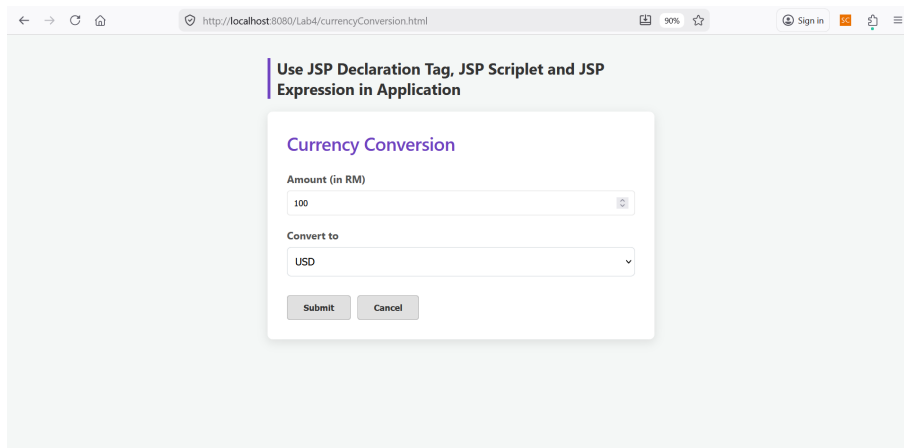


Figure 4.10: Input example in `currencyConversion.html`

4. Observe the output displayed in the browser (Figure 4.10).
5. Ensure that all output values are displayed in **2 decimal places**.

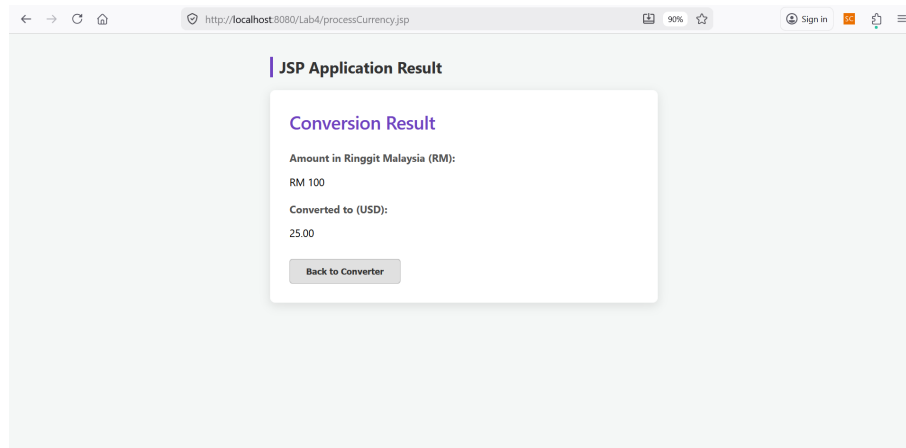


Figure 4.11: Processing result of currency conversion

Reflection

1. What have you learnt from this exercise?

4.6 Task 3: Using JSP Standard Action (Include and Param)

Objective: Apply `<jsp:include>` and `<jsp:param>` to reuse JSP components and pass data between pages.

4.6.1 Step 1: Create `jspParameter.jsp`

1. Create a new project named **Lab4**.
2. Create a new JSP file named `jspParameter.jsp`.
3. Refer to Figure 4.12 for file creation.

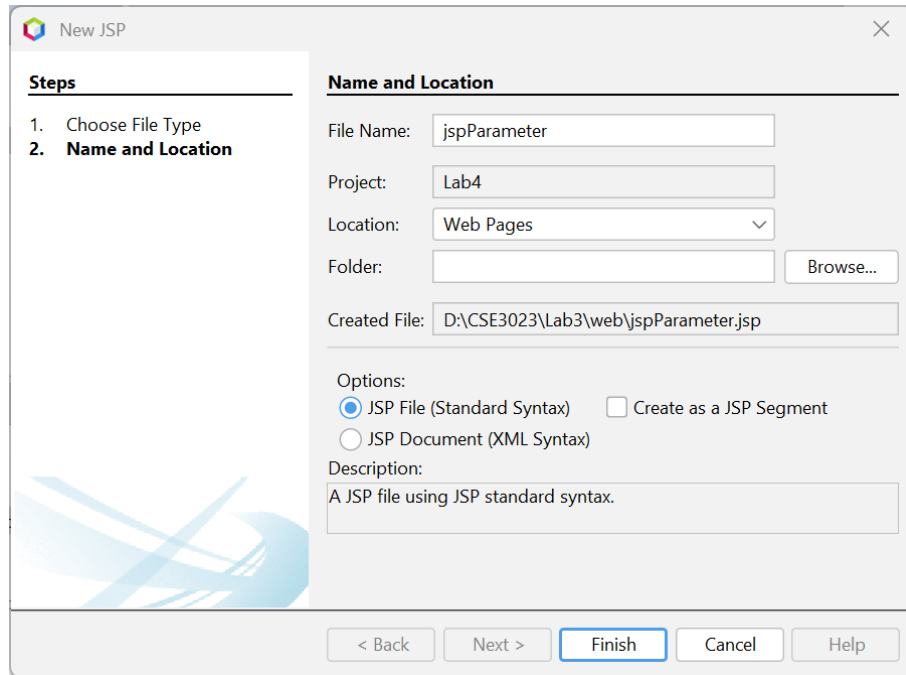


Figure 4.12: Creating jspParameter.jsp

4. Create the basic HTML structure as shown below:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Using JSP Standard Action</title>
</head>
<body>
    <h1>Using jsp:include and jsp:param to display information</h1>
</body>
</html>
```

5. Add a **JSP Scriptlet** to define subject information:

```
<%
    String sCode = "CSE3023";
    String sSubject = "Web-based Application Development";
    String sCredit = "3(2+1)";
%>
```

6. Add the **JSP Standard Action** <jsp:include> with <jsp:param> to pass data

to another JSP page:

```
<jsp:include page="subjectInfo.jsp" flush="true">
  <jsp:param name="code" value="%= sCode %>" />
  <jsp:param name="subject" value="%= sSubject %>" />
  <jsp:param name="credit" value="%= sCredit %>" />
</jsp:include>
```

7. Save the file `jspParameter.jsp`.

4.6.2 Step 2: Create `subjectInfo.jsp`

1. Create another JSP file named `subjectInfo.jsp`.
2. Refer to Figure 4.13 for file creation.

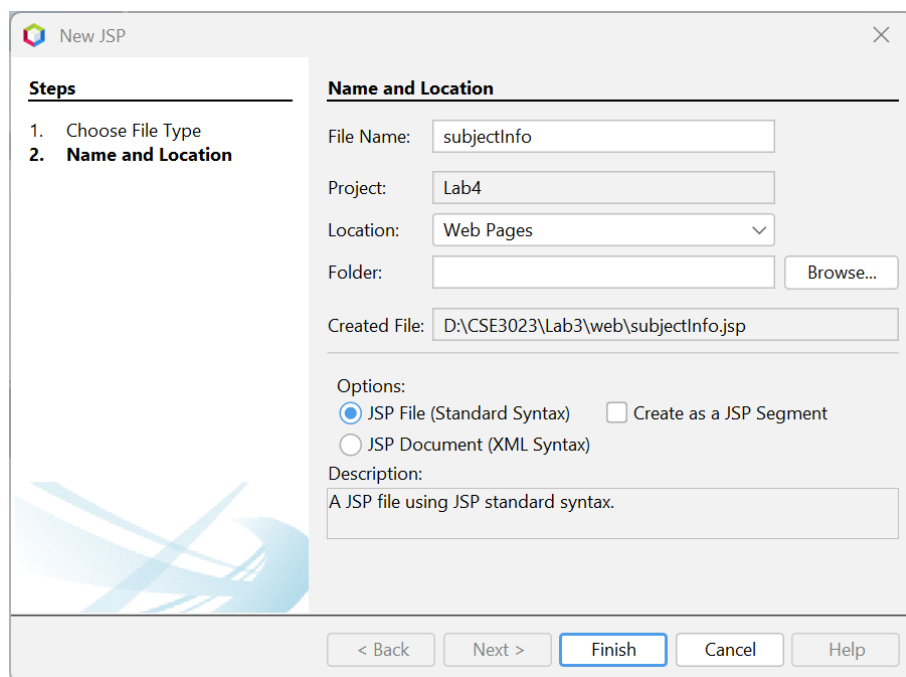


Figure 4.13: Creating `subjectInfo.jsp`

3. Create the HTML structure as shown below:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Subject Information</title>
```

```
</head>
<body>
    <h1>Subject Information</h1>
</body>
</html>
```

4. Add three (3) paragraphs and use **JSP Expression** to retrieve the parameter values passed from `jspParameter.jsp`:

```
<p>Code: <%= request.getParameter("code") %></p>
<p>Subject: <%= request.getParameter("subject") %></p>
<p>Credit: <%= request.getParameter("credit") %></p>
```

5. Save all files.

4.6.3 Step 3: Compile and Run the Application

1. Compile `jspParameter.jsp`.
2. Run `jspParameter.jsp` in a web browser.
3. Observe the output displayed as shown in Figure 4.14.

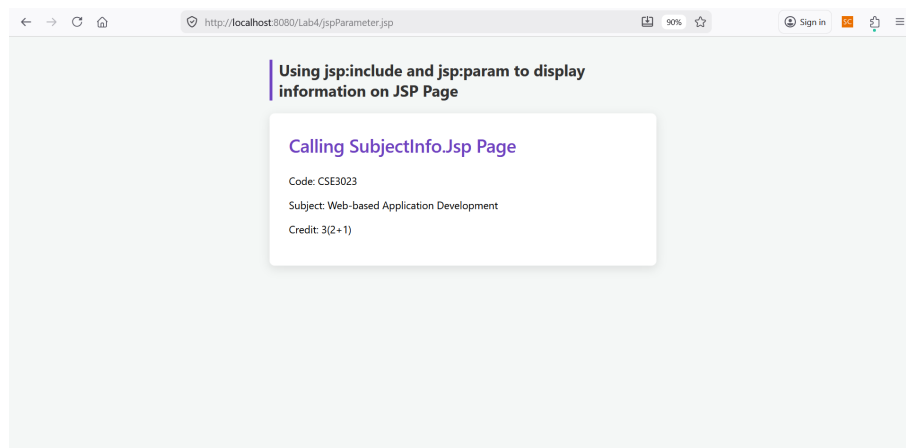


Figure 4.14: Output of `jspParameter.jsp`

Reflection

1. What have you learnt from this exercise?
2. List TWO (2) other JSP Standard Action tags.

4.7 Task 4: Using JSP Standard Action (Forward)

Objective: Apply `<jsp:forward>` and `<jsp:param>` to redirect requests and pass user data between JSP pages.

4.7.1 Step 1: Create `forward.jsp`

1. Create a new project named **Lab4**.
2. Create a new JSP file named `forward.jsp`.
3. Refer to Figure 4.15 for file creation.

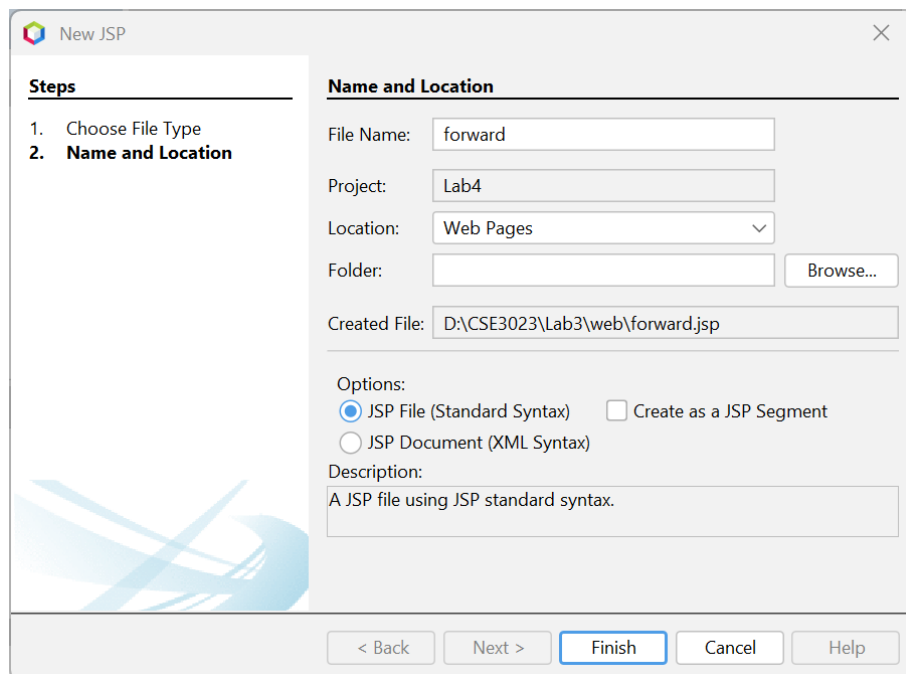


Figure 4.15: Creating `forward.jsp`

4. Create the basic HTML structure as shown below:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Using JSP Standard Action (Forward)</title>
</head>
<body>
    <h1>Using jsp:forward to display user information</h1>
```

```
</body>
</html>
```

5. Add the **JSP Standard Action** `<jsp:forward>` to redirect the request to another page and pass parameters:

```
<jsp:forward page="forwardInfo.jsp">
  <jsp:param name="uname" value="Wan Nural Jawahir Hj Wan Yussof" />
  <jsp:param name="email" value="wannurwy@umt.edu.my" />
  <jsp:param name="nationality" value="Malaysia" />
  <jsp:param name="background" value="Lecturer" />
</jsp:forward>
```

6. Save the file `forward.jsp`.

4.7.2 Step 2: Create `forwardInfo.jsp`

1. Create another JSP file named `forwardInfo.jsp`.
2. Refer to Figure 4.16 for file creation.

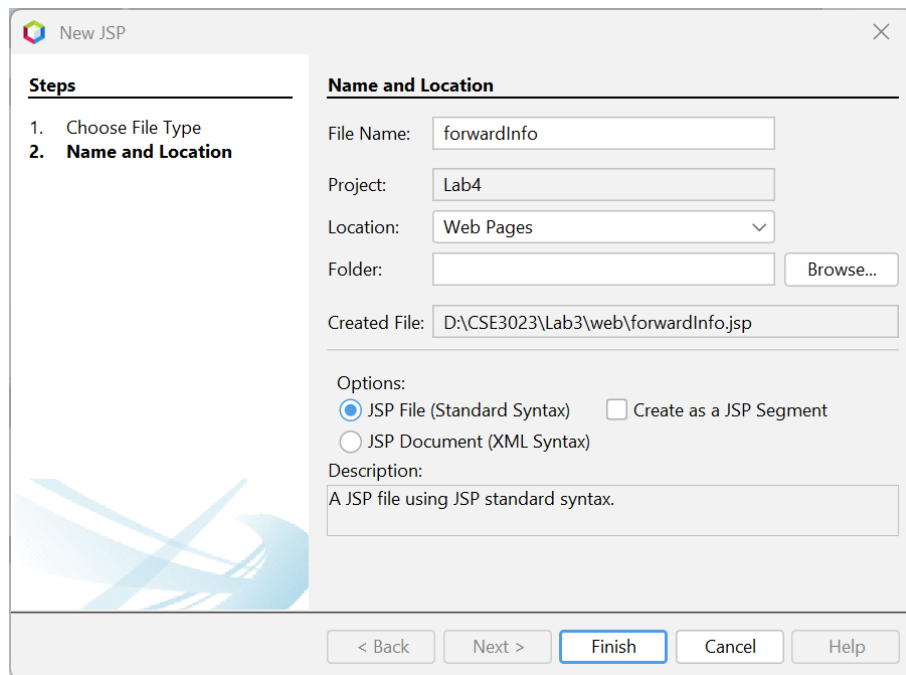


Figure 4.16: Creating `forwardInfo.jsp`

3. Use **JSP Scriptlet** to retrieve the forwarded parameters:

```

<%
    String name = request.getParameter("uname");
    String email = request.getParameter("email");
    String nationality = request.getParameter("nationality");
    String background = request.getParameter("background");
%>

```

4. Use **JSP Expression** to display the data:

```

<p>Name: <%= name %></p>
<p>Email: <%= email %></p>
<p>Nationality: <%= nationality %></p>
<p>Background: <%= background %></p>

```

5. Save all files.

4.7.3 Step 3: Compile and Run the Application

1. Compile forward.jsp.
2. Run forward.jsp in a web browser.
3. Observe that the request is automatically redirected to forwardInfo.jsp.
4. Verify that the forwarded data is displayed correctly, as shown in Figure 4.17.

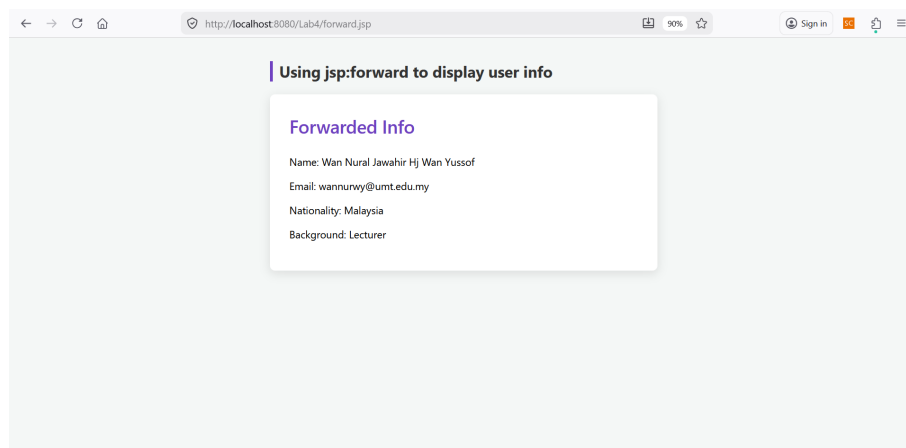


Figure 4.17: Output of forwarded information

Reflection

1. What have you learnt from this exercise?

2. List TWO (2) more JSP Standard Action Tag.

4.8 Task 5: Using JSP Scriptlet to Construct Business Logic

Objective: Apply JSP Scriptlet to implement business logic for an insurance quotation system, including calculation of insurance premium, NCD deduction, and SST charges.

4.8.1 Step 1: Create Project and JSP File `insuranceQuotation.jsp`

1. Create a new project named **Lab3**.
2. Create a new JSP file named `insuranceQuotation.jsp`.
3. Refer to Figure 4.18 for file creation.

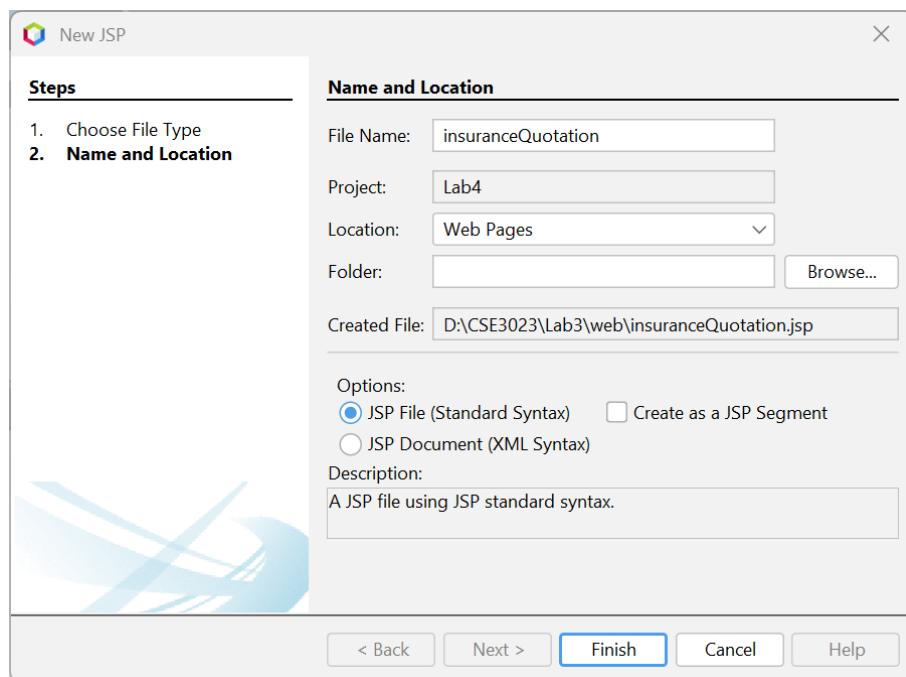


Figure 4.18: Creating `insuranceQuotation.jsp`

4.8.2 Step 2: Design Insurance Quotation Interface

1. Design the Graphical User Interface (GUI) for the insurance quotation system.
2. Ensure the interface includes all required input fields such as IC number, name, coverage type, price, and NCD.

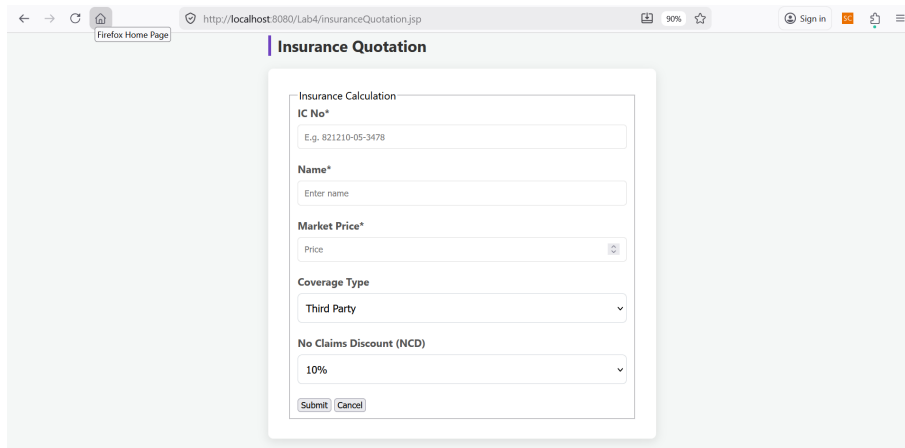


Figure 4.19: Insurance Quotation Interface

3. Refer to Figure 4.19.
4. Ensure all front-end validations (e.g., required fields and numeric validation) are implemented.

4.8.3 Step 3: Create processInsuranceQuo.jsp

1. Create another JSP file named processInsuranceQuo.jsp.
2. Refer to Figure 4.20.

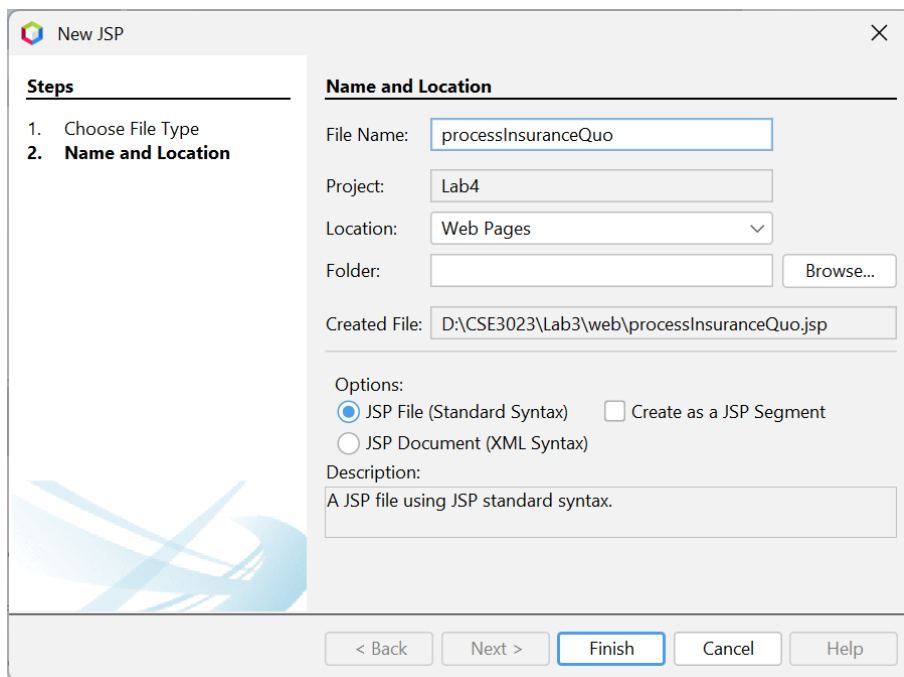


Figure 4.20: Creating processInsuranceQuo.jsp

4.8.4 Step 4: Implement Business Logic Using JSP Scriptlet

1. Implement the business logic in `processInsuranceQuo.jsp` using JSP Scriptlet.
2. The insurance calculation must include:
 - Insurance rate based on coverage type
 - NCD (No Claim Discount) deduction
 - 8% SST (Sales and Service Tax)

```
<%  
    // Retrieve form data  
    String icno = request.getParameter("icno");  
    String name = request.getParameter("name");  
    String coverage = request.getParameter("coverage");  
    String ncdStr = request.getParameter("ncd");  
  
    double price = 0;  
    double ncd = 0;  
  
    try {  
        price = Double.parseDouble(request.getParameter("price"));  
        ncd = Double.parseDouble(ncdStr);  
    } catch (Exception e) {  
        price = 0;  
        ncd = 0;  
    }  
  
    // Business Logic  
    double rate = 0;  
    String coverageDisplay = "";  
  
    if ("comprehensive".equals(coverage)) {  
        rate = 0.05; // 5%  
        coverageDisplay = "Comprehensive";  
    } else {  
        rate = 0.03; // 3%  
        coverageDisplay = "Third Party";  
    }  
  
    // Base insurance calculation
```

```
double insurance = price * rate;

// Apply NCD discount
double discount = insurance * ncd;
double afterNCD = insurance - discount;

// Add 8% SST
double sst = afterNCD * 0.08;
double finalAmount = afterNCD + sst;
%>
```

4.8.5 Step 5: Compile and Run the Application

1. Save all JSP files.
2. Right-click `insuranceQuotation.jsp` and run the project.
3. Enter test data into the system to generate insurance quotation.
4. Verify the output as shown in Figure [4.21](#).

Reflection

1. What have you learnt from this exercise?
2. List all Java features you used in Java Scriptlet.

Insurance Quotation Result

IC No:

888888-08-8888

Name:

Wan Nural Jawahir Hj Wan Yussof

Market Price (RM):

50000.00

Coverage Type:

Comprehensive

NCD:

55%

Base Insurance:

RM 2500.00

NCD Discount:

RM 1375.00

After NCD:

RM 1125.00

SST (8%):

RM 90.00

Final Insurance Amount:

RM 1215.00

[Back](#)

Figure 4.21: Insurance Quotation Output

4.8 Exercise

Objective: Develop a **BMI Calculator Web Application** by integrating JSP Scriptlets, Expressions, Page Directives, Include Directives, and JSP Standard Actions (`include`, `param`, and `forward`) as learned in this lab.

Case Study

Faculty of Computer Science and Mathematics, UMT is organizing a **Healthy Lifestyle Awareness Program**. A web-based BMI Calculator System is required to help students calculate their Body Mass Index (BMI) and determine their health category instantly.

As a web developer, you are required to develop a JSP-based application that demonstrates modular design, data passing, and dynamic processing using JSP technologies.

Instruction

Develop the system by fulfilling the following requirements:

1. Create a reusable **header.jsp** and **footer.jsp**.
2. Apply the **JSP Include Directive** (`<%@ include ... %>`) to include the header and footer in all JSP pages.
3. Design a navigation menu in the header that links to:
 - BMI Calculator Page (`bmiCalculator.jsp`)
 - Result Page
 - Health Information Page
4. Create a **BMI input form** in `bmiCalculator.jsp`:
 - Weight (kg)
 - Height (m)
5. Submit the form to `processBMI.jsp` and:
 - Retrieve values using `request.getParameter()`
 - Use **JSP Scriptlet** to:
 - Convert input values
 - Calculate BMI:
$$BMI = \frac{weight}{height^2}$$
6. Use **JSP Standard Action** `<jsp:forward>` to forward the result to `resultBMI.jsp`.
7. Pass BMI value and category using `<jsp:param>`.

8. In `resultBMI.jsp`:

- Display BMI using **JSP Expression**
- Format output to 2 decimal places
- Display BMI category:
 - BMI < 18.5: Underweight
 - $18.5 \leq \text{BMI} \leq 25$: Normal
 - BMI > 25: Overweight

9. Create `healthInfo.jsp`:

- Use **JSP Page Directive** to import Java classes
- Store BMI categories using an `ArrayList`
- Display the categories dynamically in a table

10. Ensure proper validation:

- Input must be numeric
- Height must not be zero

Assessment Rubric (10 Marks)

Criteria	Marks
System Structure: Proper JSP page organization with navigation and modular layout.	2
Include Directive: Correct use of reusable header/footer.	2
Form & Data Handling: Correct use of <code>request.getParameter()</code> .	2
Standard Actions: Proper use of <code><jsp:forward></code> and <code><jsp:param></code> .	2
Dynamic Processing: Accurate BMI calculation, classification, and display using Scriptlet & Expression.	2
Total	10